Reflective Engineering Report: Cloud Knowledge Distillation & Deployment

Student Name: Nemapu Navaneeth	**Roll Number:** 230101086
Course: CS3081 Cloud Computing	**GitHub:** https://github.com/NavaneethNemapu/cloud-command-parser

1. Project Overview & Motivation

Deploying multi-billion parameter Large Language Models (LLMs) on constrained edge devices or cost-sensitive cloud infrastructure is fundamentally restricted by memory bandwidth and hardware execution costs. This project implements sequence-level knowledge distillation (Kim & Rush, EMNLP 2016) to compress a 3-billion parameter teacher model into a lightweight 70-million parameter causal student model (EleutherAI/pythia-70m). The distilled model is subsequently containerized and deployed as a microservice on a local Kubernetes cloud cluster. The end-to-end pipeline spans parallel data labeling via SeaweedFS S3 object storage, CPU-bounded fine-tuning, zero-downtime rolling updates, horizontal pod autoscaling (HPA), stateless Network Access Control List (NACL) enforcement, and automated CI/CD quality gates.

2. Deep Technical Learnings

A. Sequence-Level Knowledge Distillation Mechanics:

Unlike token-level distillation (Hinton et al., 2015) which requires matching complete probability distribution logits over the full vocabulary at every token position (demanding enormous tensor storage and synchronization overhead), sequence-level distillation treats the teacher's greedily decoded outputs as high-quality pseudo-ground-truth targets. The student model optimizes standard cross-entropy loss over teacher-curated sequence targets, effectively transferring task-specific structural regularity without requiring access to intermediate teacher activations or expensive multi-GPU co-execution.

B. PyTorch Autograd Engine & CPU Tensor Execution:

Profiling PyTorch during the backward pass on CPU revealed that causal language models are severely compute-bound by dense matrix multiplication (GEMM) in the output projection layer (`embed_out`). Because the vocabulary dimension is large (50,304 tokens) while the hidden size is 512, computing gradients with respect to output weights involves projecting across all active sequence positions. I learned that CPU threading allocation (`torch.set_num_threads`), dynamic padding collators, and aggressive sequence truncation are essential to avoid multi-gigabyte memory thrashing.

C. S3-Compatible Object Storage & Distributed Coordination:

SeaweedFS demonstrated the decoupling of metadata operations (handled via the Filer layer using LevelDB) from physical blob chunk storage (handled by Volume Servers). Implementing independent worker shards across object storage required robust distributed coordination. I learned how atomic filesystem directory creation (`os.mkdir`) functions as a mutual-exclusion lock across independent worker processes, preventing race conditions during concurrent local and cloud storage operations.

D. Cloud-Native Reliability & Stateless Network Boundaries:

Configuring Kubernetes Deployments taught me the operational difference between liveness (`/healthz`) and readiness (`/readyz`) probes. A Pod must not receive traffic while downloading model weights from S3 during cold start. Furthermore, working with stateless NACLs reinforced that network traffic is strictly bidirectional: allowing outbound client requests is futile unless corresponding ingress rules explicitly permit return traffic on ephemeral port ranges (1024–65535).

3. Major Engineering Challenges & Implemented Solutions

Challenge 1: PyTorch CPU Backward Pass Bottleneck in Output GEMM

Diagnostic: Autograd profiling on Pythia-70M under an initial batch size of 50 and sequence length of 48 revealed that a single training step took over 15 minutes on CPU. Profiling data showed that 99.48% of total CPU backward execution time was concentrated in `aten::mm`, with 79.19% inside `MmBackward0`. With 2,400 active tokens per batch, the backward step executed dense matrix multiplication of shape (50304, 2400) @ (2400, 512), producing ~250 GFLOPs of computation that starved CPU memory channels.
Solution: I redesigned the training pipeline with dynamic collator padding, restricted `batch_size = 4`, pinned sequence length to 48, and allocated 4 dedicated intra-op CPU threads (`torch.set_num_threads(4)`). This reduced total tokens per step to 192 (a 12.5x reduction), slashing batch execution time to ~60 seconds.

Challenge 2: SeaweedFS Master Volume Saturation Across Distinct Collections

Diagnostic: When uploading trained student models to bucket `models`, SeaweedFS threw internal RPC errors: `'failed to find writable volumes for collection: models replication: 000 ttl: error: No writable volumes'`. Inspection via `weed shell volume.list` revealed the server was started with default `volume.max=8`. All 8 volume slots had been allocated to collection `datasets` and the default collection, leaving 0 free slots.
Solution: I restructured the storage startup sequence, safely unmounted empty volumes via `volume.deleteEmpty`, and configured the master server with quoted parameters `"-volume.max=50"` and `"-s3.port=8333"`. This created 42 free volume slots, enabling seamless concurrent multi-collection S3 artifact storage.

Challenge 3: Stateless NACL Symmetric Return Traffic & Rule Precedence

Diagnostic: Initial testing of the subnet access control specification showed that simulated client packets were dropped even when outbound rules allowed traffic. Unlike stateful firewalls, stateless NACLs do not track connection state.
Solution: I mapped all communication paths bidirectionally in `network/nacl.json`. For client HTTPS connections, inbound rule 210 permits port 443 while outbound rule 300 permits return traffic to client ephemeral ports (1024–65535). For backend app communication, outbound rule 100 permits port 8000 while inbound rule 400 permits return packets from the `app` subnet (`10.20.0.0/22`). Crucially, the deny rule for abusive CIDR `198.51.100.0/24` was placed at Rule 100 to ensure first-match evaluation prior to broad allow rules. Verification with `nacl_sim.py` achieved 100.0% accuracy across all test packets.

Challenge 4: Gradient Instability & Exploding Loss on Small Batches

Diagnostic: Small-batch training on causal language models is susceptible to extreme gradient variance. In early trials with a learning rate of 5e-4, loss abruptly diverged to NaN on batch 2.
Solution: I stabilized the AdamW optimizer by reducing the learning rate to `0.0001` (1e-4) and adding gradient norm clipping: `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`. This enforced strict numerical stability, yielding smooth convergence from 3.6991 (Epoch 1) to 1.5500 (Epoch 3).

4. Key Empirical Findings & Measured Observations

• **Distillation Compression vs Accuracy Retention:** The distilled Pythia-70M model reduced the parameter footprint from 3,000M (teacher) to 70M (student)—a massive **97.7% parameter reduction**. Remarkably, the student model achieved a full-frame accuracy of **59.0%** (surpassing the required 31.0% threshold) and a JSON validity rate of **97.0%**, effectively retaining 95.2% of the teacher's task performance (62.0% full-frame, 99.0% validity) while running entirely on low-power CPU hardware.
• **Memory Stability Under Strict Resource Limits:** Over 50 sequential requests under 1 vCPU and 1 GiB limits, baseline memory with model weights loaded was 458.18 MiB, while peak memory reached **554.00 MiB**. The service operated with **470.00 MiB (45.9%) of safe headroom** below the 1,024 MiB limit, exhibiting 0 out-of-memory (OOM) crashes.
• **Cold-Start Latency Breakdown:** Cold-start measurements across 3 independent runs from S3 yielded: Run 1: 10.75s, Run 2: 9.26s, Run 3: 9.24s (**Median: 9.26s**). S3 download of 6 asset files accounted for ~3.1s, while model weight deserialization and graph instantiation consumed ~6.1s.
• **High-Throughput Load Performance:** Under Locust 4-user concurrency, the dual-replica deployment sustained **32.6 requests/second** with a median latency of **120 ms** and a 95th percentile latency of **150 ms** (100x faster than the 5,000 ms ceiling) with an exact **0.0% error rate**.
• **Reliability & Zero-Downtime Updates:** During the live load test, rolling update from v1 to v2 and rollback undo executed with zero dropped requests. Pod termination recovery completed in 12.8s without triggering 500 errors.

5. Proposed Improvements & What Could Have Been Done Better

1. **Token-Level Logit Distillation with Intermediate Layer Projections:** Future iterations should combine sequence distillation with token-level KL-divergence over top-k teacher logits and hidden-state projection matrices (FitNets), improving slot boundary precision.
2. **INT8 Post-Training Quantization (PTQ) & ONNX Runtime:** Quantizing Pythia-70M weights to 8-bit integers using ONNX Runtime or PyTorch dynamic quantization would reduce memory footprint from ~280 MB to ~70 MB and cut CPU inference latency to sub-20ms.
3. **Multi-AZ Kubernetes Topology & Pod Disruption Budgets:** Moving beyond single-node clusters to multi-availability-zone deployments configured with `topologySpreadConstraints` and `PodDisruptionBudgets (minAvailable: 1)` would ensure resilience against physical host hardware failure.
4. **Dynamic Egress CIDR & eBPF Security Policies:** Replacing stateless NACLs with Cilium eBPF network policies would allow stateful L7 filtering, eliminating the requirement to expose broad ephemeral port ranges.

6. Concluding Reflection

This project provided invaluable hands-on mastery of bridging machine learning model compression with modern cloud infrastructure. I experienced firsthand the critical trade-offs between parameter scale, CPU execution latency, memory limits, and automated cloud reliability. Successfully building a reproducible, resilient, and fully verified deployment from raw data labeling to automated CI/CD quality gates has significantly deepened my competence as a systems-oriented cloud engineer.

Submission Tag: final **Verified By:** Nemapu Navaneeth (230101086)